

Ansible for z/OS

Ansible Certified Content for IBM Z





HOW WE GOT HERE

Today's IBM Z can host multiple operating systems and workloads simultaneously. These include traditional mainframe systems such as z/OS, z/VSE, z/TPF, and z/VM as well as Linux and KVM environments. In addition, the IBM mainframe can no longer operate in isolation, but must be part of a larger network environment that includes non-IBM assets.

To manage a Z system in the era of automation and heterogeneous networks, IBM needs appropriate tooling.

Ansible was founded in 2013 to provide system and network administration automation. Red Hat acquired Ansible in 2015 to augment its suite of tools for managing robust hyperconverged environments.

IBM acquired Red Hat in 2019. IBM and Red Hat had already partnered to provide solutions in that space.

IBM and Red Hat jointly developed a range of solutions to merge the IBM world and the broader world. One of these solutions is the set of Ansible collections for managing z/OS systems.

Release by release, Ansible and other Red Hat solutions are becoming more seamlessly integrated with the Z. The offerings that support this evolution are collectively called Ansible Certified Content for IBM Z.



IBM_ZOS_CORE COLLECTION



The `ibm_zos_core` collection, which you've been working with during the course, contains modules that enable Ansible to perform nearly any system administration and provisioning tasks. To summarize:

<code>zos_apf</code>	<code>zos_job_submit</code>	<code>zos_unarchive</code>
<code>zos_archive</code>	<code>zos_lineinfile</code>	<code>zos_volume_init</code>
<code>zos_backup_restore</code>	<code>zos_mount</code>	<code>zos_zfs_resize</code>
<code>zos_blockinfile</code>	<code>zos_mvs_raw</code>	
<code>zos_copy</code>	<code>zos_operator</code>	
<code>zos_data_set</code>	<code>zos_operator_action_query</code>	
<code>zos_encode</code>	<code>zos_ping</code>	
<code>zos_fetch</code>	<code>zos_replace</code>	
<code>zos_find</code>	<code>zos_script</code>	
<code>zos_gather_facts</code>	<code>zos_started_task</code>	
<code>zos_job_output</code>	<code>zos_stat</code>	
<code>zos_job_query</code>	<code>zos_tso_command</code>	

IBM_ZOSMF COLLECTION



z/OS System Management Facility (z/OSMF) is the z/OS-native facility for system administration automation. The `ibm_zosmf` collection enables Ansible to interact with z/OSMF to manage z/OS instances remotely. It consists of the following modules:

`zmf_authenticate`

`zmf_sca`

`zmf_workflows`

IBM_ZOS_SYSAUTO COLLECTION

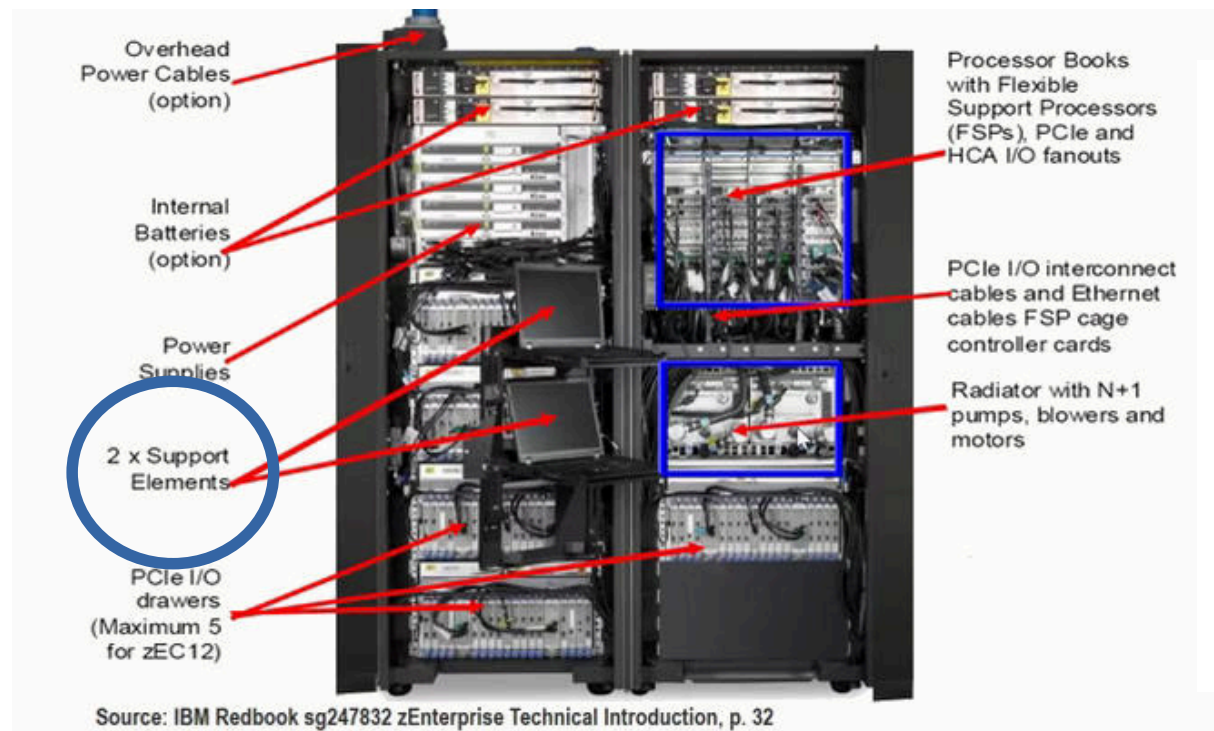
IBM Z System Automation is a Netview-based application that provides a single control point for various network administration functions. The `ibm_zos_sysauto` collection enables Ansible to connect to APIs exposed by the application so that its functions can be managed remotely. It consists of two roles:

`sa_create_dynamic_resource`

`sa_delete_dynamic_resource`



IBM_ZHMC COLLECTION



The IBM Z Hardware Management Console (HMC) is a software component on Z systems that enables low-level management of the system console or the Support Element (SE). This is the "true" system console as opposed to a TSO command line.

The `ibm_zhmc` collection enables Ansible to interact with the console. It works with systems branded IBM Z and IBM LinuxONE, which is a version of the Z without the traditional or "legacy" components.

The HMC has far too many functions to list on a slide. Different subsets of its functions are available to users that have certain security profiles – OPERATOR, ADVANCED, SYSPROG, ACSADMIN, and SERVICE.

IBM_ZOS_CICS COLLECTION

CICS (Customer Information Control System) has been around nearly as long as IBM mainframes themselves. It is one of the most widely-used traditional z/OS systems. You can think of CICS as a managed code environment and/or application container/server.

The `ibm_zos_cics` collection contains modules to support two related and somewhat overlapping areas of system administration – provisioning and dynamic resource definition.

aux_temp_storage	global_catalog
aux_trace	local_catalog
cmci_action	local_request_queue
cmci_create	region_jcl
cmci_delete	stop_region
cmci_get	td_intrapartition
cmci_update	transaction_dump
csd	

```
FLDNAME  DFHMDF POS(4,5), X
          INITIAL='I LOVE CICS', X
          LENGTH=11, X
          ATTRB=(NORM/BRT/DRK/PROT/UNPROT/ASKIP,IC,FSET,NUM,ANUM) X
          HILIGHT=HILIGHT/BLANK/REVRSE/BLINK, X
          COLOR=BLUE/RED/WHITE,ETC, X
          PICIN='X(11)', X
          PICOUT='X(11)',
```


IBM_ZOS_IMS COLLECTION



IMS (Information Management System) dates from around the same time as CICS, and serves the same general purpose. IMS includes database support as well as transaction processing.

The `ibm_zos_ims` collection enables Ansible to manage IMS assets remotely. It includes these modules:

`ims_acb_gen`

`ims_catalog_populate`

`ims_catalog_purge`

`ims_command`

`ims_dbd_gen`

`ims_dbrc`

`ims_ddl`

`ims_pcb_gen`

WHAT YOU KNOW AND WHAT YOU CAN DO

- You can use features of the `ibm_zos_core` collection to perform basic operations
- You are aware of additional IBM z/OS collections that pertain to managing assets that may not be present in all z/OS installations.
- You can navigate and use basic features of z/OS using the traditional user interface, TSO/ISPF.